

Creating Microservices for account and loan

In this hands on exercises, we will create two microservices for a bank. One microservice for handing accounts and one for handling loans.

Each microservice will be a specific independent Spring RESTful Webservice maven project having it's own pom.xml. The only difference is that, instead of having both account and loan as a single application, it is split into two different applications. These webservices will be a simple service without any backend connectivity.

Follow steps below to implement the two microservices:

Account Microservice

- Create folder with employee id in D: drive
- Create folder named 'microservices' in the new folder created in previous step. This folder will contain all the sample projects that we will create for learning microservices.
- Open <https://start.spring.io/> in browser
- Enter form field values as specified below:
 - **Group:** com.cognizant
 - **Artifact:** account
- Select the following modules
 - Developer Tools > Spring Boot DevTools
 - Web > Spring Web
- Click generate and download the zip file
- Extract 'account' folder from the zip and place this folder in the 'microservices' folder created earlier
- Open command prompt in account folder and build using mvn clean package command
- Import this project in Eclipse and implement a controller method for getting account details based on account number. Refer specification below:
 - Method: GET
 - Endpoint: /accounts/{number}
 - Sample Response. Just a dummy response without any backend connectivity.

```
{ number: "00987987973432", type: "savings", balance: 234343 }
```

- Launch by running the application class and test the service in browser

Loan Microservice

- Follow similar steps specified for Account Microservice and implement a service API to get loan account details
 - Method: GET
 - Endpoint: /loans/{number}
 - Sample Response. Just a dummy response without any backend connectivity.

```
{ number: "H00987987972342", type: "car", loan: 400000, emi: 3258, tenure: 18 }
```

- Launching this application by having account service already running
- This launch will fail with error that the bind address is already in use
- The reason is that each one of the service is launched with default port number as 8080. Account service is already using this port and it is not available for loan service.
- Include "server.port" property with value 8081 and try launching the application
- Test the service with 8081 port

Now we have two microservices running on different ports.

NOTE: The console window of Eclipse will have both the service console running. To switch between different consoles use the monitor icon within the console view.

Week 4 – Creating Microservices for Account and Loan

Objective

To create two independent Spring Boot REST microservices:

- Account Microservice
- Loan Microservice

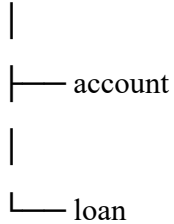
Each service runs independently on a different port and returns dummy JSON data without backend connectivity.

Technologies Used

- Java 17
- Spring Boot 4
- Spring Web

- Spring Boot DevTools
- Maven
- IntelliJ IDEA

Project Structure microservices



Account Microservice

Endpoint

GET /accounts/{number}

URL

http://localhost:8080/accounts/00987987973432

Sample Response

```
{
  "number": "00987987973432",
  "type": "savings",
  "balance": 234343.0
}
```

Loan Microservice

Endpoint

GET /loans/{number} URL

http://localhost:8081/loans/H00987987972342

Sample Response

```
{  
  "number": "H00987987972342",  
  "type": "car",  
  "loan": 400000.0,  
  "emi": 3258,  
  "tenure": 18  
}
```

Port Configuration

By default, Spring Boot applications run on 8080.

The Loan microservice was configured to use:

`server.port=8081` to

avoid port conflicts.

Result

Successfully created and tested two independent Spring Boot microservices.

- Account Service running on Port 8080
- Loan Service running on Port 8081

Both REST APIs returned the expected JSON responses.



Project
☐ Gradle - Groovy
☐ Gradle - Kotlin
☒ Maven

Language
☒ Java
☐ Kotlin
☐ Groovy

Spring Boot
☐ 4.1.1 (SNAPSHOT)
☒ 4.1.0
☐ 4.0.8 (SNAPSHOT)
☐ 4.0.7

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☐ 21 ☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web WEB
Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

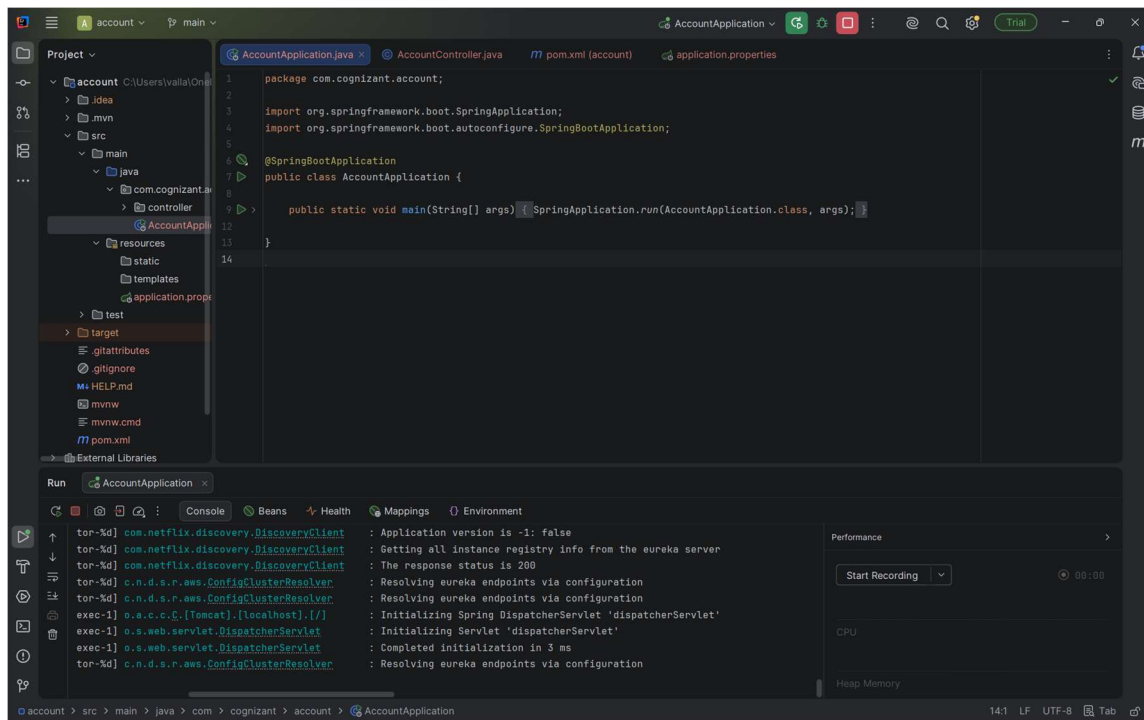
Spring Boot DevTools DEVELOPER TOOLS
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.



GENERATE CTRL + G

EXPLORE CTRL + SPACE

...



localhost:8080/accounts/123

{ "number": "123", "type": "savings", "balance": 234343 }

spring initializr

Project

☐ Gradle - Groovy
 ☐ Gradle - Kotlin
 ☒ Maven

Language

☒ Java
 ☐ Kotlin
 ☐ Groovy

Spring Boot

☐ 4.1.1 (SNAPSHOT)
 ☒ 4.1.0
 ☐ 4.0.8 (SNAPSHOT)
 ☐ 4.0.7

Project Metadata

Group

com.cognizant

Artifact

loan

Package name

com.cognizant.loan

Packaging

☒ Jar
 ☐ War

Configuration

☒ Properties
 ☐ YAML

Java

☐ 26
 ☐ 25
 ☐ 21
 ☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot DevTools

DEVELOPER TOOLS

Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

loan

main

LoanApplication

LoanController

LoanApplication.java

src

main

java

com.cognizant.loan

controller

LoanController

LoanApplication

resources

static

templates

application.properties

test

target

gitattributes

gitignore

HELP.md

mvnw

mvnw.cmd

pom.xml

External Libraries

Scratches and Consoles

application.properties

pom.xml (loan)

LoanController.java

LoanApplication.java

```

1 package com.cognizant.loan;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class LoanApplication {
8
9     public static void main(String[] args) { SpringApplication.run(LoanApplication.class, args); }
10
11 }
12
13
14

```

Run

LoanApplication

Console

Beans

Health

Mappings

Environment

```

] o.s.c.n.e.s.EurekaServiceRegistry : Registering application LOAN with eureka with status UP
] com.netflix.discovery.DiscoveryClient : Saw local status change event StatusChangeEvent [timestamp=1784199792989, currentStatus=UP, previousStatus=DOWN]
] com.netflix.discovery.DiscoveryClient : DiscoveryClient_LOAN/SRAVYA:loan:8081: registering service...
] o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8081 (http) with context path '/'
] o.s.c.n.e.s.EurekaAutoServiceRegistration : Updating port to 8081
] com.netflix.discovery.DiscoveryClient : DiscoveryClient_LOAN/SRAVYA:loan:8081 - registration status: 204
] com.cognizant.loan.LoanApplication : Started LoanApplication in 7.362 seconds (process running for 9.315)
] c.n.d.s.r.aws.ConfigClusterResolver : Resolving eureka endpoints via configuration
] c.n.d.s.r.aws.ConfigClusterResolver : Resolving eureka endpoints via configuration

```

Performance

Start Recording

00:00

Suggested plugin Reactive Streams available.

Configure plugins...

Don't suggest again

loan > src > main > java > com > cognizant > loan > LoanApplication

14:1 LF UTF-8 Tab

```
{ "number": "H00987987972342", "type": "car", "loan": 400000, "emi": 3258, "tenure": 18 }
```



HOME LAST 1000 SINCE STARTUP

System Status

Environment	test	Current time	2026-07-16T16:33:31 +0530
Data center	default	Uptime	00:05
		Lease expiration enabled	false
		Renews threshold	6
		Renews (last min)	4

EMERGENCY! EUREKA MAY BE INCORRECTLY CLAIMING INSTANCES ARE UP WHEN THEY'RE NOT. RENEWALS ARE LESSER THAN THRESHOLD AND HENCE THE INSTANCES ARE NOT BEING EXPIRED JUST TO BE SAFE.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ACCOUNT	n/a (1)	(1)	UP (1) - SRAVYA:account:8080
LOAN	n/a (1)	(1)	UP (1) - SRAVYA:loan:8081

General Info

Name	Value
total-avail-memory	100mb
num-of-cpus	16
current-memory-usage	46mb (46%)
server-uptime	00:05
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/ ,
available-replicas	

Instance Info

Name	Value
ipAddr	192.168.50.86
status	UP